

Week 1 Discussion

...

CSE29

We need your GitHub account for PAs



Fill out this sheet with your GitHub account (e.g., `aschulm` in `github.com/aschulm`)

Attendance!!!



[https://forms.gle/JZULqxevoBJPYxa
M6](https://forms.gle/JZULqxevoBJPYxaM6)

Overview: UTF-8 Encoding

Background: Word Encoding

- What is word encoding?
 - Taking human readable text and sometimes special characters and symbols like emojis, and transforming them into binary bits for the computer to read
- Why do we use word encodings?
 - A computer can't read words plainly, it only processes things in 1's and 0's
 - The same way you might not be able to understand a language someone speaks, so you need it translated to one you can understand

Hi



01001000
01100101

Text → Binary



Dog

01100100
01101111 01100111

Background: ASCII

- ASCII is a method of word encoding like UTF-8
 - Was popular and widely used for a long time before UTF-8 came out
 - Limited in the number of characters it could represent
- ASCII was great for the entire English alphabet, as well as numbers, and important characters
- Encodes characters in 1 byte (8 bits)
 - 0XXXXXXXX
 - The 0 is the starting bit that lets you know this is an ASCII character encoding, so the other 7 bits were how characters were encoded
- Could only represent $2^7 = 128$ characters (0 to 127)

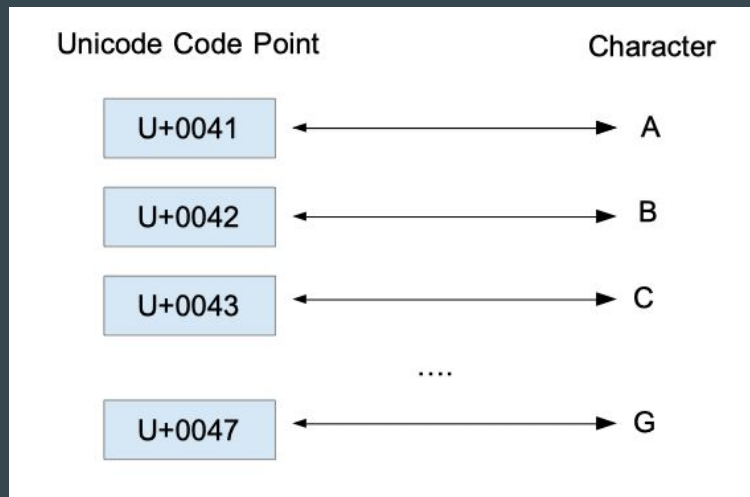
CONVERT TEXT TO BINARY

English Letter	ASCII Decimal	Binary String
d	100	01100100
o	111	01101111
g	103	01100111

01100100 01101111 01100111 = dog

Unicode Codepoints

- Unique numerical value used to represent a specific character, symbol, or control code
- The notation is “U+” followed by a hexadecimal value
 - U+0041 = “A”
- The range is from U+0000 to U+10FFFF
 - So ~1.1 million possible different characters
 - ASCII is the range U+0000 to U+007F
- These are NOT encodings, just a unique identification number (think like your PID/TSN)



A new foe has appeared!

UTF-8 Encoding

CHALLENGER APPROACHING

UTF-8 Encoding

- UTF-8 encoding solves the problem of limited number of unique bit combinations for encodings that ASCII had
- UTF-8 allows encoding of characters that take up to 21 bits to encode them
 - Also covers the entire range of unicode codepoint
- Uses variable length encoding allowing encodings for 1, 2, 3, and 4 byte characters
 - This is possible using a prefix header for each encoding based on the number of bytes
 - 1 byte - 0XXXXXX (Same as ASCII)
 - 2 bytes - 110XXXXX
 - 3 bytes - 1110XXXX
 - 4 bytes - 11110XXX
- Notice how for 2 through 4, the number of 1's in the prefix = number of bytes
 - This is how a computer knows how many bytes an encoding will be

UTF-8 Cont.

- How do we encode the rest of the bytes?
 - Ex: 110XXXXX ...
 - What comes after that first 8 bits (1 byte)?
- That's where continuation prefix headers come in
 - Every subsequent byte corresponding to the encoding has the prefix 10 followed by the remaining 6 bits of encoding
 - Ex: 110XXXXX 10XXXXXX (This is a full 2 byte encoding in UTF-8)

Byte 1	Byte 2	Byte 3	Byte 4
0yyyyzzzz			
110xxxxyy	10yyzzzz		
1110www	10xxxxyy	10yyzzzz	
11110uvv	10vvwww	10xxxxyy	10yyzzzz

UTF-8 Cont.

- Each byte length has a range of codepoints that it can/should represent
 - I say “should” because technically a longer length like 4 bytes can represent all shorter lengths, but that is very wasteful memory wise
- Here are the ranges

Code point ↔ UTF-8 conversion

First code point	Last code point	Byte 1	Byte 2	Byte 3	Byte 4
U+0000	U+007F	0yyyyzzzz			
U+0080	U+07FF	110xxxxyy			
U+0800	U+FFFF	1110wwww	10xxxxyy	10yyzzzz	
U+010000	U+10FFFF	11110uvv	10vvwwww	10xxxxyy	